DAY 09 | POST-READ

Mongo APIs and JavaScript: Recap and Beyond

What we covered today, plus ideas to stretch your understanding.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

Part A: Summary of today

Topic	What to remember
Mongo CRUD API	insert_one, find, find_one, update_one with $set, delete_one. Validate ObjectId. Return str ids.
Aggregation	A pipeline of stages: $match, $group, $sort, $project, $unwind, $limit.
Accumulators	$sum, $avg, $min, $max, $push inside $group.
Offset pagination	skip((page - 1) * size).limit(size), plus total count.
Cursor pagination	Filter by _id greater than the last seen id; fast for feeds.
Indexes	create_index on filtered and sorted fields; compound for filter + sort; unique for no duplicates.
JS variables	const by default, let when reassigning, avoid var.
JS functions	function name() {} or arrow functions (a, b) => a + b.
JS collections	Arrays with map, filter, reduce; objects with keys; destructuring.
JS equality	Always === and !==.

Part B: Going beyond today

1. $lookup: joining collections

```
{"$lookup": {"from": "customers", "localField": "customer_id",
             "foreignField": "_id", "as": "customer"}}
```

Use sparingly. If you $lookup constantly, consider embedding the data instead.

2. Text search

```
products.create_index([("name", "text"), ("description", "text")])
products.find({"$text": {"$search": "wireless keyboard"}})
```

3. Async MongoDB drivers

PyMongo is synchronous. For high-traffic async FastAPI apps, the PyMongo async API or Motor lets endpoints await database calls.

4. reduce in JavaScript

```
const total = [120, 450, 80].reduce((sum, p) => sum + p, 0);   // 650
```

5. Spread and optional chaining

```
const updated = { ...user, city: "Osaka" };   // copy with a change
const zip = order?.address?.zip;              // undefined instead of an error
```

6. JSON in JavaScript

```
const text = JSON.stringify({ a: 1 });   // to a string
const obj = JSON.parse(text);            // back to an object
```

Part C: Check yourself

- What do $match and $group do in an aggregation pipeline?
- When is cursor pagination better than skip and limit?
- Why can too many indexes be a problem?
- What is the difference between const and let?
- Why should you prefer === over ==?

Part D: Practice ideas for tonight

- Add a /stats/top-customers endpoint using $group and $limit.
- Write a JavaScript function that groups an array of orders by status into an object.
- Rewrite three of your Day 2 Python exercises in JavaScript and run them with Node.js.

Coming up on Day 10: promises, async and await, npm, building a CRUD API with Express, and testing APIs with Postman.

Otsukaresama deshita (お疲れ様でした): thank you for your hard work today